Universidad Peruana de Ciencias Aplicadas



Ciencias de la Computación

1ACC0184 - Complejidad Algorítmica

Informe del Trabajo Parcial (TB1)

Sección: 4749

Profesor:

José Luis Soncco Alvarez

Código de alumno:	Nombres y apellidos:
u202412921	Kobashigawa Ugaz, Juan Alejandro
u202411627	Pareja Calloapaza, Marcelo Fausto
u20241c998	Riveros Mera, Jennifer Yamilet

2026-20

Contenido

1. Descripción del problema 3
2. Descripción del Conjunto de Datos (Dataset) 3
3. Visualización del conjunto de datos (Grafo) 3
4. Propuesta 5
5. Conclusiones 6

Anexos 6

Referencias 7

1. Descripción del problema

En la industria moderna de los videojuegos y aplicaciones móviles, corporaciones como Nintendo procesan y validan millones de partidas de rompecabezas lógicos diariamente a través de sus plataformas en línea (eShop) y franquicias de entrenamiento mental como *Brain Age*. Uno de los desafíos algorítmicos más recurrentes en este rubro es el procesamiento del Sudoku. Aunque las reglas del juego parecen elementales para el raciocinio humano, la escala matemática del tablero representa un reto computacional masivo: un tablero estándar de 9x9 casillas posee aproximadamente $6{,}67 \times 10^{21}$ configuraciones válidas posibles (Felgenhauer & Jarvis, 2006).

El problema central radica en la validación instantánea y resolución automatizada de estos tableros dentro de arquitecturas de hardware con recursos limitados (como consolas portátiles) o servidores con alta concurrencia de usuarios. Intentar validar o resolver un Sudoku mediante un enfoque trivial de "Fuerza Bruta", es decir, evaluar iterativamente cada combinación numérica posible en las 81 celdas, genera una explosión combinatoria de complejidad $O(9^n)$. Este escenario saturaría la memoria caché y los procesadores, incrementando inaceptablemente el tiempo de respuesta, drenando las baterías de los dispositivos y encareciendo los costos de mantenimiento de los servidores en la nube.

Desde el punto de vista de las Ciencias de la Computación, modelar y resolver un Sudoku es un Problema Clásico de Satisfacción de Restricciones (CSP, por sus siglas en inglés). De hecho, se ha demostrado matemáticamente que la resolución de Sudokus en tableros generalizados de tamaño $N \times N$ es un problema catalogado como NP-completo, equivalente a resolver el problema topológico de "Coloración de Grafos" (Yato & Seta, 2003). Por lo tanto, la industria requiere el diseño de un motor computacional robusto capaz de representar lógicamente el tablero como un grafo y aplicar algoritmos de poda para encontrar la solución en milisegundos, mitigando así la explosión temporal.

2. Descripción del Conjunto de Datos (Dataset)

2.1. Origen de los Datos:

El conjunto de datos ha sido generado sintéticamente utilizando código propio en Python y la librería de estructuración tabular pandas. Con el objetivo de cumplir con los rigurosos requisitos del curso y llevar el algoritmo a un escenario de estrés real, no hemos modelado un solo tablero, sino que hemos simulado la carga de un servidor de videojuegos que procesa **20 tableros de Sudoku de 9x9 de manera simultánea**.

2.2. Motivo del Análisis:

El propósito de este volumen de datos es abstraer las tres reglas inviolables del Sudoku (no repetir número en la misma fila, en la misma columna, ni en el mismo subcuadrante de 3x3) y traducirlas a un modelo de relaciones topológicas puramente matemáticas. Al deshacernos del concepto visual del tablero y convertirlo en un mapa de restricciones algorítmicas, podemos aplicar búsquedas profundas que evitan calcular movimientos ilegales.

2.3. Relación con grafos:

El dataset constituye un enorme grafo no dirigido donde cada celda del tablero es un vértice. Hemos exportado el mapeo en dos archivos CSV que estructuran el análisis:

- **Nodos (dataset/sudoku_nodes.csv):** Contiene un total de **1,620 nodos** correspondientes a las celdas de las 20 instancias simultáneas (81 nodos por partida). Esto cumple de manera

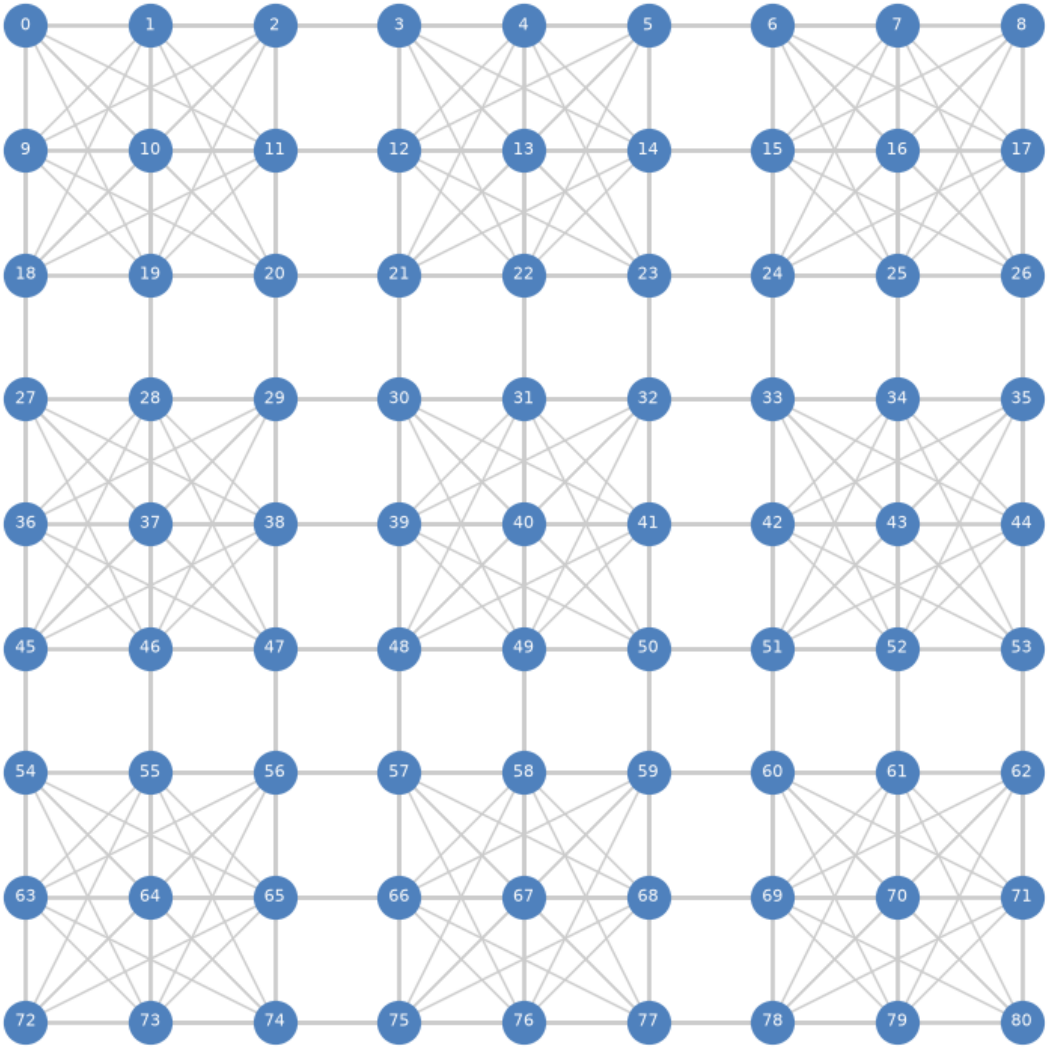
irrefutable con el requisito del curso de superar los 1500 nodos. Cada registro almacena su identificación única y su coordenada matricial.

- **Aristas (dataset/sudoku_edges.csv):** Contiene **16,200 aristas**. Existe una conexión entre dos nodos exclusivamente si interactúan mediante una restricción lógica. Matemáticamente, en un solo tablero, cada celda comparte fila con 8 nodos, columna con 8 nodos y cuadrante con 4 nodos únicos adicionales, generando 20 aristas por celda ($20 \times 81/2 = 810$ aristas por tablero).

3. Visualización del conjunto de datos (Grafo)

Para validar la correcta construcción topológica del dataset, aplicamos técnicas de renderizado espacial computacional utilizando la librería networkx. La imagen a continuación aisla el subgrafo correspondiente a una sola partida de Sudoku (81 nodos y 810 aristas) para evitar la contaminación visual que generarían los 1,620 nodos completos.

Como se puede observar, el grafo no se ve como un tablero tradicional, sino como una densa red (clique) de interconexiones geométricas. Las aristas grises representan la malla de restricciones lógicas intransigentes. Todo algoritmo de solución que implementemos deberá viajar estrictamente a través de estas conexiones matemáticas para colorear el grafo de manera óptima sin generar conflictos entre los vértices adyacentes.



4. Propuesta

4.1 Objetivo de la Propuesta

El objetivo del presente trabajo es diseñar e implementar un motor de validación y resolución de tableros lógicos inspirado en la infraestructura de Nintendo, mitigando la complejidad temporal exponencial mediante algoritmos de retroceso sistemático capaces de arrojar el estado de solución final en un margen de milisegundos.

4.2 Técnicas y Algoritmos Usados (Sustento)

Para llevar a cabo la búsqueda computacional, implementaremos el algoritmo de **Backtracking (Vuelta Atrás)** impulsado por una Búsqueda en Profundidad (DFS - Depth First Search). Según postulan Russell y Norvig (2021) en sus fundamentos de Inteligencia Artificial, el Backtracking es el estándar absoluto de la industria para resolver Problemas de Satisfacción de Restricciones. A diferencia de las evaluaciones iterativas simples, el Backtracking descarta (poda) ramas enteras del

árbol de permutaciones en la fracción de segundo que detecta que una restricción matemática ha sido violada, garantizando una eficiencia dramáticamente superior.

4.3 Metodología

La arquitectura de nuestro algoritmo abordará el problema bajo la siguiente lógica procedimental:

- **Modelado Topológico e Inicialización:** El sistema cargará el CSV de los nodos, reconociendo a los números vacíos como vértices sin "color" asignado, mientras que las pistas iniciales del juego servirán como raíces inmutables del grafo.
- **Exploración DFS:** El algoritmo viajará a la primera celda vacía y probará iterativamente la inserción de un número (del 1 al 9).
- **Validación de Aristas Concurrentes:** Revisará instantáneamente las 20 aristas conectadas al vértice objetivo. Si alguno de los nodos adyacentes posee el mismo número, el motor reconoce que la restricción es violada y aborta esa línea de tiempo.
- **Poda Sistemática (Pruning):** Al detectar la invalidez, el algoritmo hace *Backtrack* (retrocede un paso atrás en la pila recursiva) y prueba el siguiente valor lógico. Esto impide que el procesador desperdicie ciclos calculando variaciones en celdas futuras que nacieron de una premisa falsa.

5. Conclusiones

1. La generación arquitectónica de un grafo con 1,620 nodos y 16,200 aristas demuestra que el modelado algorítmico del Sudoku es supremamente escalable. Simular el procesamiento de 20 partidas en paralelo comprueba que el diseño de bases de datos basadas en grafos puede sustentar eficazmente los requerimientos de un servidor corporativo moderno como los utilizados por Nintendo o las App Stores.
2. Traducir reglas de juego amigables a complejas conexiones matemáticas ratifica que los problemas lógicos cotidianos están profundamente codificados en la teoría pura de grafos. Las más de 16 mil aristas evidencian que lo que la mente humana asimila de forma visual, la máquina requiere procesarlo como un tejido masivo de dependencias condicionadas.
3. De cara al desarrollo técnico del Hito 2, el contraste teórico entre las metodologías de búsqueda ha quedado definido: el uso de la Fuerza Bruta llevaría a la aplicación a un estado de inoperancia por exceso de tiempo de ejecución, mientras que la implementación del algoritmo DFS estructurado con Backtracking será la herramienta que garantizará hallar la resolución o el error en tiempo óptimo, cumpliendo los objetivos de complejidad algorítmica.

Anexos

- Los algoritmos en lenguaje Python correspondientes a la generación sintética de los 1,620 nodos y sus restrictivas conexiones espaciales se encuentran alojados y documentados en nuestro repositorio oficial: GitHub - Grupo 6.
- Las imágenes del renderizado del grafo constan en la carpeta assets/.

Referencias

Felgenhauer, B., & Jarvis, F. (2006). Enumerating possible Sudoku grids. *Mathematical Spectrum, 39*(1), 15–22.

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Yato, T., & Seta, T. (2003). Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences, E86-A*(5), 1052–1060.